

PROGRAMMING II

DOCUMENTATION AND UNIT TESTING

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.Y. 2025/2026

DOCUMENTATION



AUTOMATED DOCUMENTATION

Documenting the code enhances code understanding and maintainability

- ▶ For you and your collaborators
- ▶ For anyone that want use your code

Documentation can be **automatically generated** by using javadoc

- ▶ The programmer writes the documentation as code comments
- ▶ The javadoc tool automatically generates web pages
- ▶ Write documentation faster and in a standard way

AUTOMATED DOCUMENTATION (CONT'D)

The javadoc tool understands **doc comments** only

- ▶ They are enclosed within `/**` and `*/`
- ▶ Multiple lines allowed, starting with `*`
- ▶ They can contain HTML tags

```
/**  
 * This is a doc comment to feed javadoc with  
 */
```

1
2
3

DOCUMENTATION TAGS

The javadoc tool supports **tags** to parse doc comments

- ▶ They start with **@** and are case sensitive
- ▶ They must be at the beginning of the line

```
/** 1
 * Print the kth element of a String array 2
 * 3
 * @author Tony Stark 4
 * @param arr The array whose element is to be printed 5
 * @param k The index of the element to be printed 6
 */ 7
public void printK(String[] arr, int k) { System.out.println(arr[k]); } 8
```

DOCUMENTATION TAGS (CONT'D)

Important tags

- ▶ `@param`: describes a specific method parameter
- ▶ `@return`: describes the return value of a method
- ▶ `@throws`: describes the exceptions a method may throw
- ▶ `@link`: provides a pointer to a class
- ▶ `@deprecated`: provides an explanation of why the code was deprecated

DOCUMENTING A CLASS

To document a class just put a doc comment on top of it

```
/** 1
 * A class modeling university students 2
 * 3
 * It is a subclass of the {@link pkg.Person} class 4
 * @author Bruce Banner 5
 */ 6
public class Student extends Person { 7
    // fields and methods 8
} 9
```

DOCUMENTING A METHOD

To document a method just put a doc comment on top of it

```
/** 1
 * <p>A method computing the square of a number 2
 * <a href="https://en.wikipedia.org/wiki/Square_(algebra)">See more</a> 3
 * </p> 4
 * 5
 * @param num The number to raise to the power two 6
 * @return The square of the given number 7
 * @since 1.0 8
 */ 9
public int computePow(int num) { 10
    return pow * pow; 11
} 12
```


DOCUMENTING VARIABLES

To document a variable just put a doc comment on top of it

```
/**  
 * The public name of a person  
 */  
public String name;  
  
/**  
 * The private age of a person  
 */  
private int age;
```

1
2
3
4
5
6
7
8
9

Private fields are not documented, unless the `-private` option is passed to `javadoc`

GENERATING THE DOCUMENTATION

By using the command line

```
javadoc -d output packages/classes
```

To generate the docs for all packages and classes in the current directory

```
javadoc -private -d doc *
```

Web pages are saved in the doc directory, private fields and methods considered

- Flags must be added for some tags, like @author and @since

GENERATING THE DOCUMENTATION (CONT'D)

By using a build tool like Maven with a suitable plugin

```
<build>                                <!-- to add to the pom.xml Maven configuration file -->
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-javadoc-plugin</artifactId>
      <version>3.6.2</version>
      <configuration>
        <source>1.8</source>
        <show>private</show>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Run with `mvn javadoc:javadoc`

UNIT TESTING



TESTING

Programmatic way of checking **software quality**

- ▶ Check that code behaves as expected
- ▶ Help during code development
- ▶ Enforce consistency during code evolution
- ▶ Improve code understanding
- ▶ Reduce debugging time

// test-driven development

// regression testing

Testing can be **manual** or automated

- ▶ Automated testing can be used to spot bugs or vulnerabilities

TESTING (CONT'D)

A test case represents a scenario for running code, comprising

- ▶ An **input** for the code *// values, actions, events*
- ▶ An **expected output** for the code

A test case succeeds if the expected output is produced in response to an input

- ▶ Typically, isolated components are tested (e.g., a class)

UNIT TESTING IN JAVA

JUnit is a framework for writing **unit tests** of Java code

- ▶ Implemented within Java, by using specific classes and annotation

Test a single Java class

- ▶ A test case tests a single method of the class
- ▶ A test suite is a collection of test cases

UNIT TESTING IN JAVA (CONT'D)

JUnit runs a test suite and reports the results

For each test case in the test suite

- ▶ JUnit calls the method from the test cases
- ▶ If the method terminates without errors, the test case is passed

Technically, a test case is a method with no return value

- ▶ If the test fails, the method throws an `AssertionFailedError`
- ▶ The JUnit framework catches the error and transparently deals with it

ASSERTIONS

The package `org.junit.jupiter.api` contains the `Assertions` class

// JUnit 5

The class provides methods to specify assertions

```
static void assertTrue(condition, message)
```

If *condition* is evaluated to

- ▶ **true**: execution continues to the following instruction
- ▶ **false**: execution jumps to the end of the method (and *message* is displayed)
- ▶ The error message is optional

ASSERTIONS (CONT'D)

The package `org.junit.jupiter.api` contains the `Assertions` class *// JUnit 5*

The class provides methods to specify assertions

```
static void assertEquals(expected, actual, message)
```

If *expected* is equal to *actual* *// works for primitive and object values*

- ▶ Then execution continues to the following instruction
- ▶ Else execution jumps to the end of the method (and *message* is displayed)
- ▶ The error message is optional

ASSERTIONS (CONT'D)

The package `org.junit.jupiter.api` contains the `Assertions` class *// JUnit 5*

The class provides methods to specify assertions

```
static void assertEquals(expected, actual, message)
```

If *expected* is equal to *actual* *// works for primitive and object arrays*

- ▶ Then execution continues to the following instruction
- ▶ Else execution jumps to the end of the method (and *message* is displayed)
- ▶ The error message is optional

FIRST UNIT TEST

```
import org.junit.jupiter.api.Test;           1
import org.junit.jupiter.api.BeforeAll;      2
import org.junit.jupiter.api.Assertions;      3

                                              4
public class StudentTest { // append 'Test'   5

                                              6

    private static Student student; // object from the class to test 7

                                              8

    @BeforeAll // executed at first           9
    void static setup() { student = new Student("Sam", 123); }    10

                                              11

    @Test                                         12
    public void testGetMatricola() { // prepend 'test'           13
        int actual = student.getMatricola();    14
        Assertions.assertEquals(123, actual, "Exp: 123, Act: " + actual); 15
    }                                             16
}                                              17
```

FIRST UNIT TEST (CONT'D)

To simplify code, it is possible to use `static` imports

When a class provides a `static` method

- Import that method only, avoiding the fully-qualified name when using it

```
import static org.junit.jupiter.api.Assertions.assertEquals; 1
... 2
@Test 3
public void testGetMatricola() { 4
    int actual = student.getMatricola(); 5
    assertEquals(123, actual, "Exp: 123, Act: " + actual); 6
} 7
8
```

JUNIT ANNOTATIONS

@DisplayName	Custom display name for a test class or a test method
@BeforeEach	The annotated method will be executed before each test method
@AfterEach	The annotated method will be executed after each test method
@BeforeAll	The annotated method will be executed before all test methods
@AfterAll	The annotated method will be executed after all test methods
@Disabled	Disables a test class or method

PROJECT ORGANIZATION

Test classes are typically appended with Test

- ▶ A different folder is typically used for test classes

Maven and Gradle naming convention

- ▶ `src/main/java` contains the project source classes
- ▶ `src/test/java` contains the project test classes

Test should be package-consistent

- ▶ A test class `some.pkg.ClazzTest`
- ▶ Should correspond to a source class `some.pkg.Clazz`

RUNNING THE TESTS

By using a build tool like Maven with a suitable plugin

```
<dependencies>          <!-- to add to the pom.xml Maven configuration file -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>5.10.0</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>3.5.4</version>
  </dependency>
</dependencies>
```

Run with `mvn test`

Q + A

